

Week 2: Digital data

MY472: Data for Data Scientists

https://lse-my472.github.io/

Autumn Term 2026

Ryan Hübert

Associate Professor of Methodology

Slides last updated: 6 October 2026

Outline

1 Digital data

2 Digital data in Python

3 File I/O

4 Character encoding

5 Tokenisation

What is digital data?

Data is a representation of information

- **Digital data** represents information using discrete values

In modern computers, digital data is represented as **bits**

- Bit is short for **binary digit**: it takes two values, either `0` or `1`
- With n bits 2^n patterns can be represented
- E.g., 2 bits gives four possibilities: `00`, `01`, `10`, `11`

Bytes are an aggregation of bits: 8 bits = 1 byte

- kilobytes (KB), megabytes (MB), gigabytes (GB), etc. are metric aggregations of bytes (but there is some ambiguity)
- $\neq$ kilobits (Kb), megabits (Mb), gigabits (Gb), etc.

Bytes in binary, decimal and hex

A single byte can represent up to $2^8 = 256$ pieces of information

Bytes can be written in multiple ways:

1. **Binary** or **bin** (base 2) writes the bits directly, e.g. `00101010`
   - The "natural" way of writing bytes since a bit is either `0` or `1`
2. **Decimal** or **dec** (base 10) gives each bit pattern an integer from 0–255, e.g. `42`
3. **Hexadecimal** or **hex** (base 16) writes each 4 bits as one of `0`–`9`, `a`–`f`, e.g. `2a`
   - Hex codes are the most common way to write bytes, often marked like `0x2a` or `\x2a`

Number systems

What does it mean to say “base 2”, “base 10”, “base 16”?

These refer to different **number systems**

- Base 2 only two numbers, `0` and `1`
- Base 10 only has ten numbers, `0`, `1`, `2`, …, `9`
- Base 16 only has sixteen “numbers”, `0`, `1`, …, `9`, `a`, …, `f`

This seems a little unusual at first because the only number system we typically use for mathematics is base 10

You can always convert in Python, e.g. a (fake) LSE candidate number of 27382 is in base 10 and can be converted to base 16:

```
hex(27382)
```

```
'0x6af6'
```

Storage and memory

Computers store and process digital data

- Computer chips contain billions of transistors that act like tiny switches
- These switches represent two logical states: `0` or `1`

On computers, digital data lives in two main places:

1. **Storage** keeps data even with the power off
   - Examples: SSDs, USB drives, hard drives, CDs/DVDs
2. **Random access memory (RAM)** holds data currently being used
   - More RAM lets a computer work with more data at once

Creating, reading and deleting files

You create digital data by making it yourself, or by copy/paste from somewhere else (locally, or over a network)

When you “open” a file, you **read** it from storage into memory

- **File Input/Output (I/O)** refers to reading data from and writing data to storage
- Keep in mind: storage $\neq$ memory

When you “delete” a file, the device marks its bits as **stale**

- Stale bits are (eventually) overwritten by new data
- Until then, “deleted” data can be recovered: use **secure erase** and/or encrypt your drive (next week)

Outline

1 Digital data

2 Digital data in Python

3 File I/O

4 Character encoding

5 Tokenisation

Python objects

In Python, digital data is packaged into **objects**, each of which has a type

Object type	Python	Example	Mutable?
Bytes	`bytes`	`b"hello"`	No
String	`str`	`"hello"`	No
Float	`float`	`3.14`	No
Integer	`int`	`42`	No
Logical	`bool`	`True`	No
List	`list`	`[1, 2]`	Yes
Tuple	`tuple`	`(1, 2)`	No
Dictionary	`dict`	`{"a": 1}`	Yes

Some object types are **mutable** and can be changed in place, while others are **immutable** (more on this later)

Python objects

We can **assign** (or **bind**) an object (the number 2.4) to the name `z`:

```
z = 2.4   # assign an object
type(z)   # what type of object is it?
```

```
<class 'float'>
```

If executed in console, `z` is in Python's **global namespace**

- Under the hood: `float` object is digital data occupying memory, and `z` is a **reference** to it recorded in the global namespace

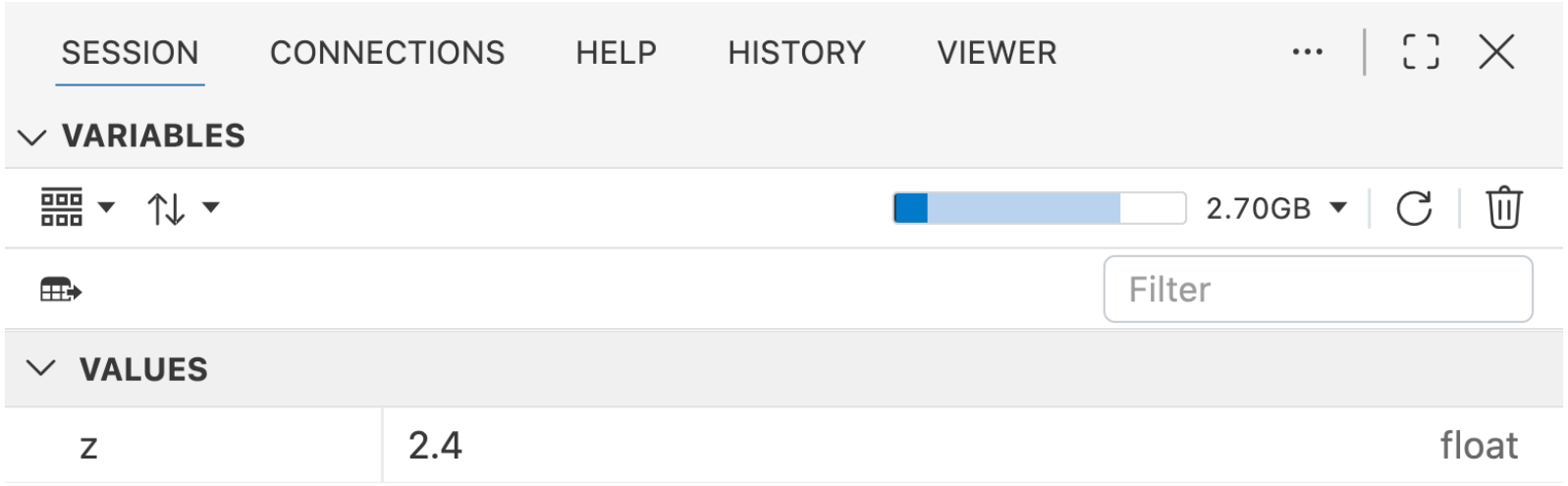
Objects can have multiple references, or **aliases**

```
w = z           # `w` and `z` now refer to the same object
print(w is z)   # verify that w is the same object as z
```

```
True
```

Python objects

You can see in the VARIABLES pane of Positron:



Inspecting an object

More programmatic way to see what's in Python's global namespace:

```
"z" in globals() # is the name "z" in global namespace?
```

```
True
```

```
globals()["z"]   # what object is assigned to the name?
```

```
2.4
```

Since Python objects are just bytes in memory, you can see how many bytes an object is:

```
import sys
z_size = sys.getsizeof(z) # in bytes
print(z_size)
```

```
24
```

Inspecting an object

You can also look at the raw bytes

1. Find the object's location in memory:

```
z_address = id(z)
print(z_address)       # In dec, get hex with hex(z_address)
```

```
5036232816
```

2. Look at the `z_size` bytes starting at that location

```
import ctypes
z_bytes = ctypes.string_at(z_address, size = z_size)
print(z_bytes.hex(" "))  # as formatted hex codes
```

```
04 00 00 00 00 00 00 00 90 cf 51 2d 01 00 00 00 33 33 33 33 33 33 03 40
```

(Side note: some code only works for CPython implementation, check what you have with `sys.implementation.name`)

Deleting an object

Use `del` to remove a name from the global namespace:

```
del z               # removes name `z` from global namespace
"z" in globals()    # check the name "z" is gone
```

```
False
```

The Python object is (mostly) untouched

- Remember: the alias `w` still refers to it, so it's still there:

```
print(w)
print(ctypes.string_at(z_address, size = z_size).hex(" "))
```

```
03 00 00 00 00 00 00 00 90 cf 51 2d 01 00 00 00 33 33 33 33 33 33 03 40
```

- One tiny change: first byte went from `03` to `02` (Why?)

Deleting a *name* is not deleting an *object*

Deleting an object

When the last reference to a specific object disappears, then the memory block is **deallocated**

```
del w                # the last reference is now gone
```

The bytes at that address are no longer reserved for the object

- What happens next is unpredictable
- Your old value is *eventually* overwritten, but you can't guarantee when

Mutability

Mutable objects keep same location in memory after changes, immutable objects are *copied* to a new location when altered

```
x = "Hybert"                # immutable
print(id(x))
```

```
5037007824
```

```
x = x.replace("y", "u") # change letter y -> u
print(id(x))
```

```
5037013488
```

```
y = [1,3]                   # mutable
print(id(y))
```

```
5038549376
```

```
y[1] = 4                    # change value 3 -> 4
print(id(y))
```

```
5038549376
```

Outline

1 Digital data

2 Digital data in Python

3 File I/O

4 Character encoding

5 Tokenisation

All files are stored as bytes

We've just looked at digital data in memory (specifically in Python)

But all digital data in files is stored as bytes too

The file `myfile.ryan` in `week02` contains 9 bytes (written in hex):
`4C 53 45 20 4D 59 34 37 32`

You can read raw bytes into Python using `open()` and `mode = "rb"` ("read binary"):

```
ryan_file = os.path.join(wdir, "myfile.ryan")
with open(ryan_file, mode = "rb") as f:
  ryan_byte_data = f.read()
print(ryan_byte_data.hex(" ")) # print as nicely formatted hex
```

```
4c 53 45 20 4d 59 34 37 32
```

File I/O in Python

You can also look at the bytes in other formats:

- In decimal (integers):

```
print(list(ryan_byte_data))    # print as dec
```

```
[76, 83, 69, 32, 77, 89, 52, 55, 50]
```

- In binary (bits):

```
[format(x, "08b") for x in list(ryan_byte_data)]
```

```
['01001100', '01010011', '01000101', '00100000', '01001101',
'01011001', '00110100', '00110111', '00110010']
```

File I/O in Python

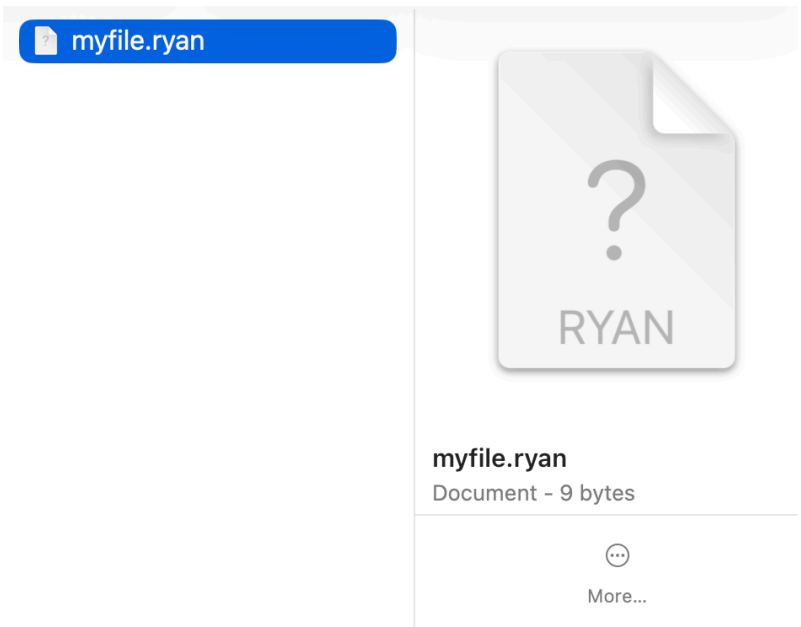
A file is exactly as big as its number of bytes:

```
print(ryan_byte_data.hex(" ")) # print as nicely formatted hex
```

```
4c 53 45 20 4d 59 34 37 32
```

```
len(ryan_byte_data)            # print number of bytes
```

```
9
```



Interpreting bytes

Bytes in `myfile.ryan` could represent anything:

- text “LSE MY472”
- integers 76, 83, 69, 32, 77, 89, 52, 55, 50
- part of an image, a colour, a sound, etc.

Software reading bytes assume a rule about how to interpret bytes

- It infers a rule from the file extension (such as `.qmd`), the **file signature** in the first few bytes, or metadata
- In this case: the file extension `.ryan` doesn’t indicate anything, and the file itself has no metadata (it might have “magic” bytes)

Interpreting bytes

Digital data is always **machine-readable** because machines can read and interpret bytes

With an encoding and software that can decode it, digital data can become **human-readable**

- Files that are not human readable are often called **binary** files
- Most of the time, when someone says a file is "machine-readable" they are implying *not* human-readable

There are "degrees" of human-readability, but at a minium, a human-readable file must be *text-based*

Interpreting bytes

We can ask Python to turn the bytes into readable text:

```
ryan_data_readable = ryan_byte_data.decode() # convert bytes to text
print(ryan_data_readable)
```

```
LSE MY472
```

What is going on here?

- With `mode = "rb"`, Python did minimal interpreting—it returned the raw bytes
- `.decode()` applied a rule to turn those bytes into characters

The key question: how did Python know how to convert *those* bytes into *that* readable text?

Outline

1 Digital data

2 Digital data in Python

3 File I/O

4 Character encoding

5 Tokenisation

Characters and strings

A **character** is the “smallest component of written language that has semantic value”

- See `https://unicode.org/glossary/#character`
- For example, “A”, “ü”, “2”, etc.

A **(character) string** is a sequence of characters

- Strings are at the core of all of data science
- We’ve already worked with them quite a bit
- Python stores strings as `str` objects

Character encoding

A **character encoding** defines how a machine "translates" bits/bytes of data into strings

It takes two things to do that:

1. A **character set**: a list of characters, each with an ID number
    - **Code points** are the unique ID numbers associated with characters in a character set
    - These can be expressed in multiple formats
2. An encoding: a rule for turning those numbers into bytes

There are different encodings, using differing numbers of bits to represent characters, e.g. 7-bit, 8-bit, 16-bit, etc.

ASCII

ASCII: the original character set/encoding, uses 7 bits

- Could only encode up to $2^7 = 128$ characters… not enough!

ASCII was later extended to 8 bits, e.g. ISO-8859-1

- Now could encode $2^8 = 256$ characters… still not enough!

Full tables here: original ASCII, ISO-8859-1

Example: hex code point for "A" is 41 with bits: `1000001` (original) `01000001` (extended)

As you can imagine: different languages, different characters → different character sets and encodings

A mess… see `http://en.wikipedia.org/wiki/Character_encoding`

Unicode and UTF encodings

Unicode is a character set that defines a code point for every character in every writing system

- **UTF-8**, **UTF-16** and **UTF-32**: different encodings for Unicode
- Note on terminology: ASCII is a character set *and* encoding

UTFs differ by how many bytes they use:

- UTF-8 is **variable width encoding**: 1, 2, 3 or 4 bytes
- UTF-16 mostly uses 2 bytes, sometimes 4 bytes
- UTF-32 is a **fixed width encoding** using 4 bytes
- UTF-16 and UTF-32 involve **null bytes** (`00000000` or `00` in hex)

UTF-8 is now the default character encoding for most digital data

UTF-8 examples

Char	Code point	Byte 1	Byte 2	Byte 3	Byte 4
&	U+0026	00100110			
u	U+0075	01110101			
ü	U+00FC	11000011	10111100		
Д	U+0414	11010000	10010100		
€	U+20AC	11100010	10000010	10101100	
	U+1FAE0	11110000	10011111	10101011	10100000

UTF-8 examples

Char	Code point	Byte 1	Byte 2	Byte 3	Byte 4
&	U+0026	26			
u	U+0075	75			
ü	U+00FC	C3	BC		
Д	U+0414	D0	94		
€	U+20AC	E2	82	AC	
	U+1FAE0	F0	9F	AB	A0

UTF-16 examples

Char	Code point	Byte 1	Byte 2	Byte 3	Byte 4
&	U+0026	26	00		
u	U+0075	75	00		
ü	U+00FC	FC	00		
Д	U+0414	14	04		
€	U+20AC	AC	20		
🫠	U+1FAE0	3E	D8	E0	DE

Note: these are **little endian** where null bytes go last (instead of **big endian**)

UTF-32 examples

Char	Code point	Byte 1	Byte 2	Byte 3	Byte 4
&	U+0026	26	00	00	00
u	U+0075	75	00	00	00
ü	U+00FC	FC	00	00	00
Д	U+0414	14	04	00	00
€	U+20AC	AC	20	00	00
	U+1FAE0	E0	FA	01	00

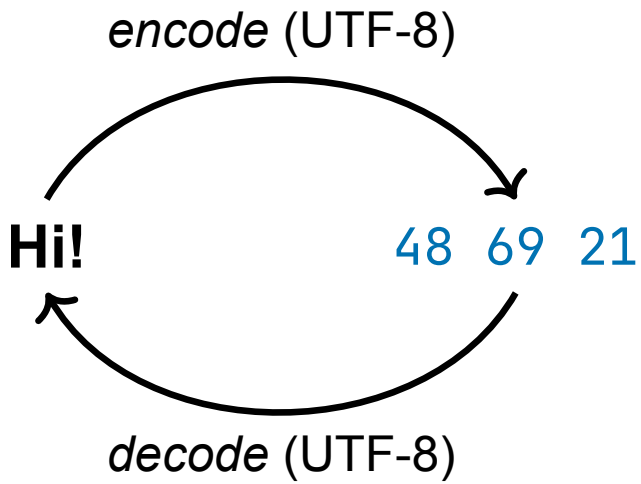
Note: these are **little endian** where null bytes go last (instead of **big endian**)

Encoding and decoding

In the context of character encodings:

- You **encode** when you go from characters to bytes
- You **decode** when you go from bytes to characters

Character encodings are lossless and reversable



Character encoding in Python

In Python, text is typically stored as a `str` object, which are sequences of Unicode code points (one for each character)

```
ord("ü") # gives the Unicode code point for a character
```

```
252
```

```
chr(252) # gives the character for a Unicode code point
```

```
'ü'
```

```
[ord(x) for x in "Hübert"] # Unicode code points for a full string
```

```
[72, 252, 98, 101, 114, 116]
```

Unicode reports code points in hex with at least four digits:

```
"U+" + f"{ord("ü"):X}".zfill(4)
```

```
'U+00FC'
```

Character encoding in Python

To get bytes from characters, encode with a *specific encoding* (UTF-8 is the default):

```
print("ü".encode().hex(" "))    # UTF-8
```

```
c3 bc
```

```
u16 = "ü".encode("utf-16-le")
print(u16.hex(" "))             # UTF-16
```

```
fc 00
```

Decoding only works with the same encoding:

```
print(u16.decode("utf-16-le"))  # back to "ü"
```

```
ü
```

```
u16.decode()                    # Python assumes UTF-8: error
```

```
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xfc in position
0: invalid start byte
```

Encoding choice changes file size

Encoding changes the size (in bytes) of the same text:

```
text = "Ryan Hübert"
len(text.encode("utf-8"))     # UTF-8 bytes
```

```
12
```

```
len(text.encode("utf-32-le")) # UTF-32 bytes
```

```
44
```

- UTF-8: 1 byte per ASCII character, more only when needed
- UTF-32: always 4 bytes per character
- So for mostly-English text, UTF-8 files are far smaller

```
len(text.encode("utf-32")) # UTF-32 bytes
```

```
48
```

- In this case: a four-byte **byte-order mark (BOM)** is put at front

Potential encoding issues

Wrongly detected encoding

- Encoding is *not* stored as metadata in plain text files
- Software guesses which encoding is used
- A wrong guess leads to import errors and corrupted characters, known as mojibake

UTF-8 is now widely used, but other encodings are still in use

- Some parts of Windows use UTF-16, while Unix-based platforms mostly use UTF-8
- Some applications (e.g. MS Office) may use other encodings, such as Windows-1252
- Some computers use a different default encoding, e.g. if bought in a country not using Latin-based characters

File encodings in Python

When you use `mode = "r"`, it decodes raw bytes into a `str` object, using UTF-8 unless you say otherwise:

```
with open(ryan_file, mode = "r", encoding = "utf-8") as f:
  my_text = f.read()
my_text
```

```
'LSE MY472'
```

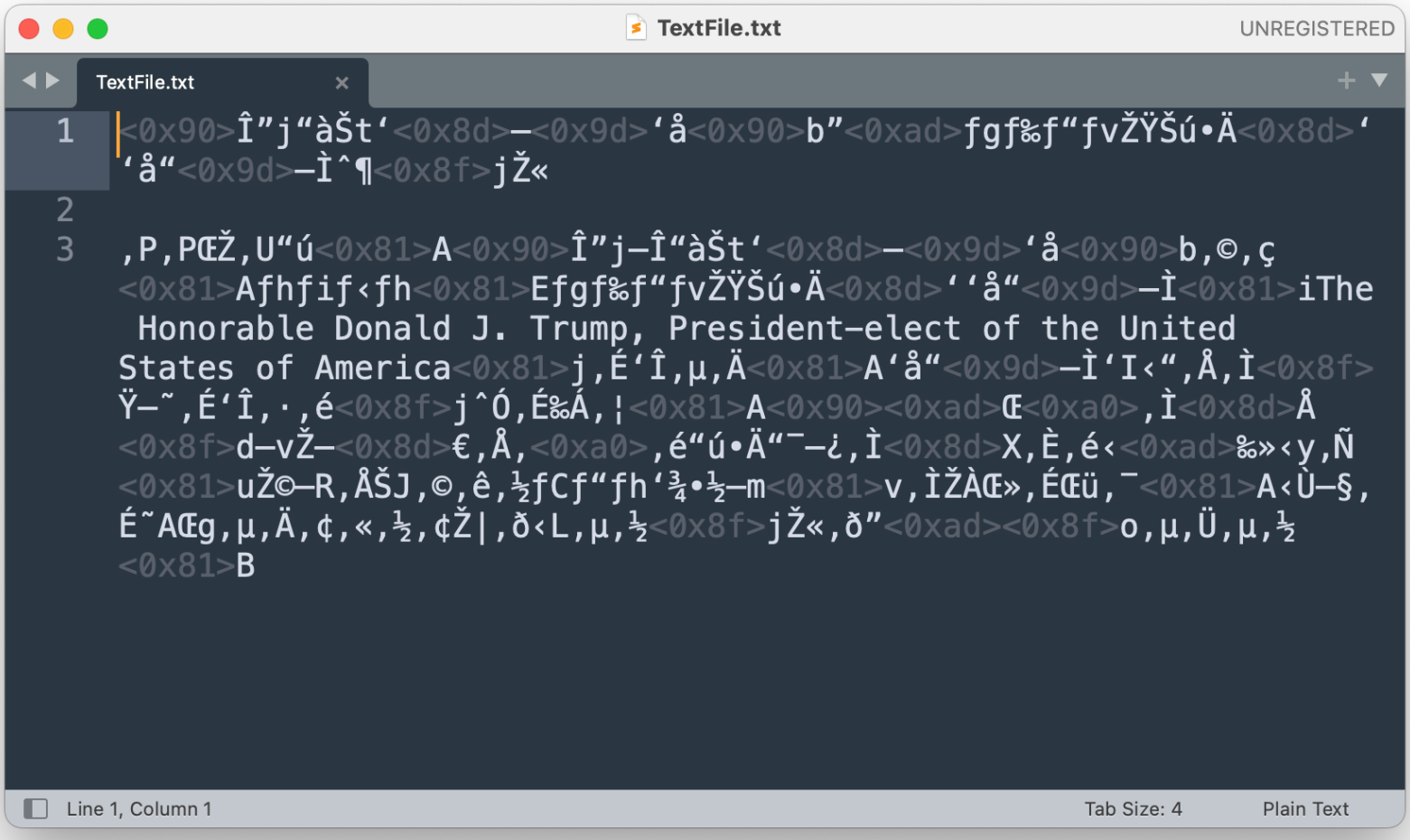
That only works for UTF-8 encoded characters:

```
unk_path = os.path.join(wdir, "TextFile.txt")
with open(unk_path, mode = "r", encoding = "utf-8") as f:
  unk = f.read()
```

```
UnicodeDecodeError: 'utf-8' codec can't decode byte 0x90 in position
0: invalid start byte
```

File encodings in Python

Not just a Python problem! Text editor also assumes wrong encoding and shows mojibake:



File encodings in Python

What can you do? First load the file as raw bytes:

```
with open(unk_path, mode = "rb") as f:
  unk = f.read()
print(unk[0:18].hex(" ")) # print first 18 bytes in hex
```

```
90 ce 94 6a 93 e0 8a 74 91 8d 97 9d 91 e5 90 62 94 ad
```

Again: you can always read the bytes stored in any file directly into Python without any encoding

Use the `charset_normalizer` module to guess encoding:

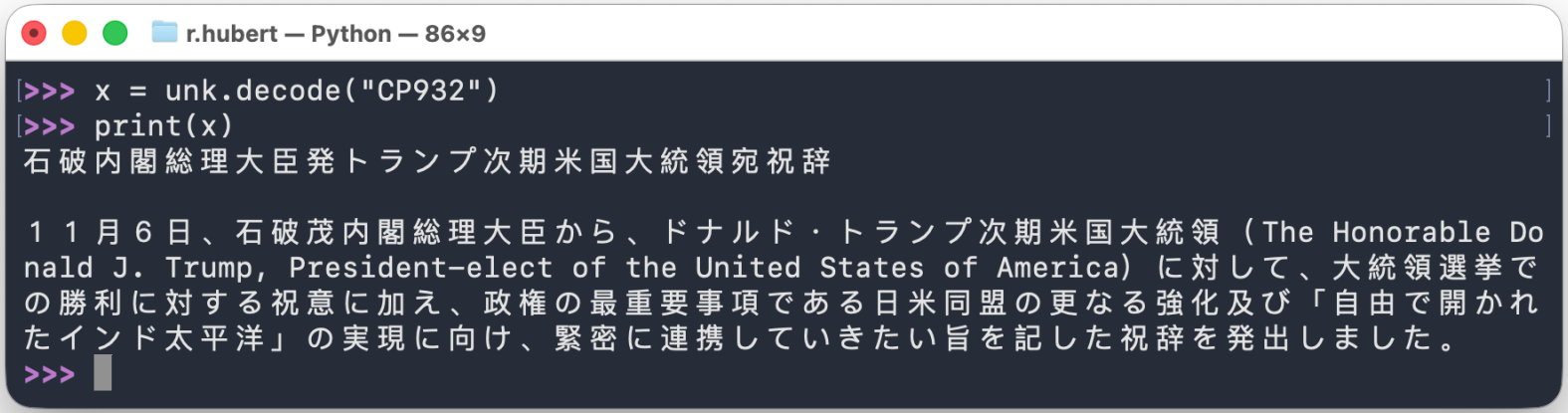
```
import charset_normalizer
charset_normalizer.detect(unk)
```

```
{'encoding': 'CP932', 'language': 'Japanese', 'confidence': 0.899}
```

Here it guesses CP932, a Japanese-specific encoding

File encodings in Python

Then, if you know encoding (or Python's guess seems promising) you can decode the bytes:

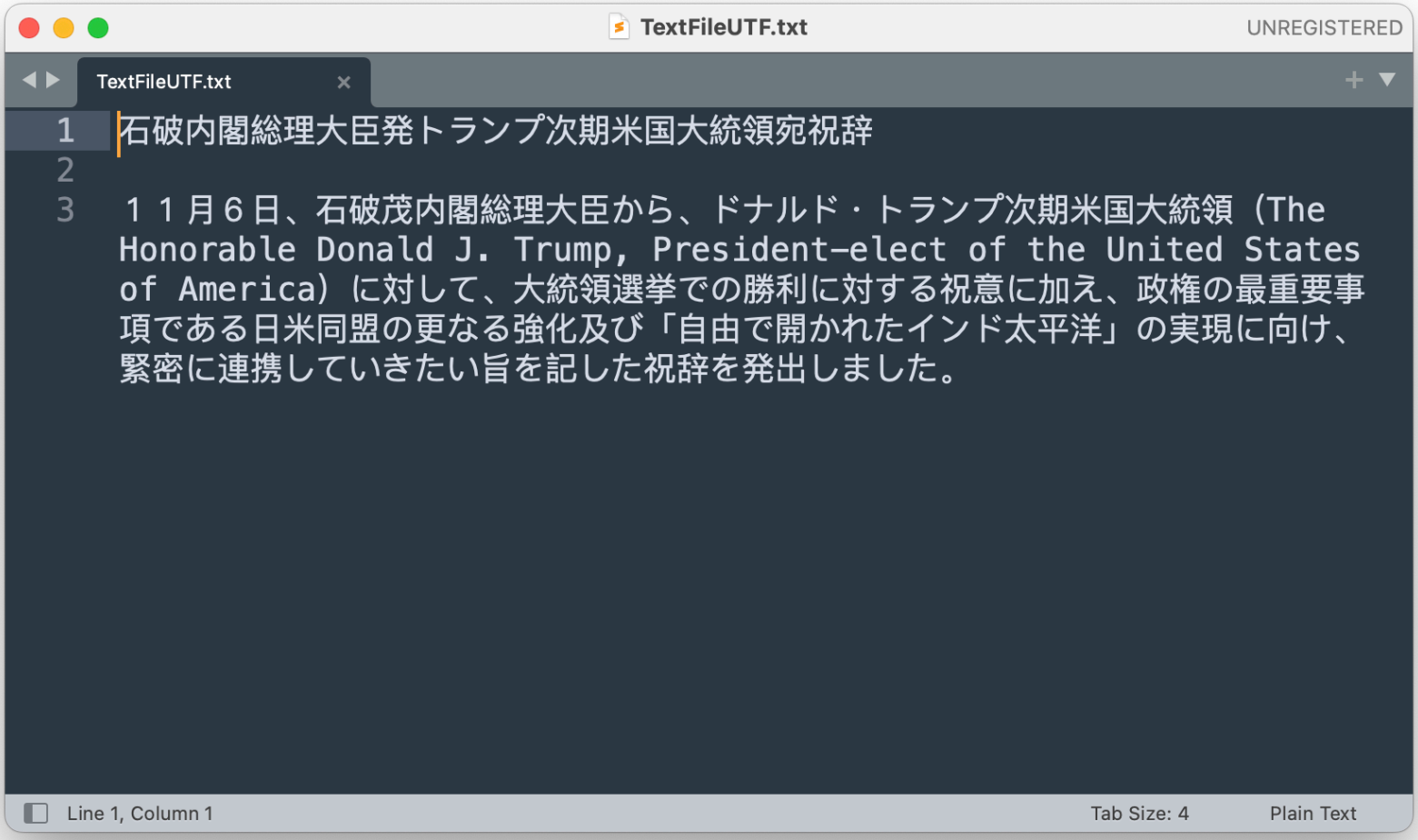


When satisfied, save the resulting text as UTF-8 for future

```
utf8_path = os.path.join(wdir, "TextFileUTF8.txt")
with open(utf8_path, mode = "w", encoding = "utf-8") as f:
  f.write(x)
```

Note: `encoding = "utf-8"` makes sure the file is written as UTF-8 encoded bytes (but it probably not necessary)

File encodings in Python



Python's `bytes` object

Look again at bytes from `TextFile.txt` with and without formatting

```
unk[0:18].hex(" ")  # str object
```

```
'90 ce 94 6a 93 e0 8a 74 91 8d 97 9d 91 e5 90 62 94 ad'
```

```
unk[0:18]           # bytes object
```

```
b'\x90\xce\x94j\x93\xe0\x8at\x91\x8d\x97\x9d\x91\xe5\x90b\x94\xad'
```

This is a special Python object type called a `bytes` object

- It prints like a string, but with a `b` before the initial quote mark
- Displays bytes using a combination of hex codes and plain text
- Hex codes are written with a `\x`, so `\x1a` is the hex code `1A`
- When a specific byte can be decoded to a *printable* 7-bit ASCII character, then the character is printed instead

Python's `bytes` object

It is pedagogically annoying that `bytes` objects mix hex codes and ASCII characters

- This is why I have been showing you `.hex(" ")` until now, but I could have also shown you in decimal:

```
print(list(unk[0:10]))    # first 10 in decimal
```

```
[144, 206, 148, 106, 147, 224, 138, 116, 145, 141]
```

The mixed display can mislead you:

```
unk[0:18]
```

```
b'\x90\xce\x94j\x93\xe0\x8at\x91\x8d\x97\x9d\x91\xe5\x90b\x94\xad'
```

The `j`, `t` and `b` aren't real characters—they're bytes that just *happen* to land in the printable ASCII range

Why use plain text?

A **plain text** file is nothing but bytes that decode to characters

- No hidden formatting, nothing you cannot read yourself (assuming your software accurately guesses the encoding!)
- Plain text is stable, persistent across time—less data corruption
- Plain text: `.txt`, `.py`, `.csv`, `.json`—and **mark up** files, which store formatting as plain text "tags" (`.md`, `.qmd`, `.html`)
- Not plain text: `.docx`, `.xlsx`, `.pptx`

Some data science advice:

- Use plain text files and stop using Word (and similar)
- Use software that encodes in UTF-8 by default
- Always choose UTF-8 when asked (e.g. when downloading)
- Know that "rich text" word processors silently change your characters, e.g. a straight `"` (U+0022) to a curly `“` (U+201C)
- If you want formatting, do it at end (e.g., compiling a `.qmd` file)

Not all files are plain text

Many Word documents look like nothing but text, but try reading one as text:

```
doc_path = os.path.join(wdir, "my_document.docx")
with open(doc_path, mode = "r") as f:
  doc = f.read()
```

```
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xd2 in position
16: invalid continuation byte
```

A `.docx` is *not* a plain text file

- `mode = "r"` tries to decode the bytes as UTF-8 and cannot
- Not an encoding problem, no encoding would fix it
- How do I know this? The file type: characters are in there somewhere, but buried under layers of other things
- Of course: can still read the bytes if we want with `mode = "rb"`

Outline

1 Digital data

2 Digital data in Python

3 File I/O

4 Character encoding

5 Tokenisation

Tokens

Character encodings allow humans to read data on computers

But computers also need to be able to read data we supply

- Narrow sense: our typed characters get encoded into bytes

Language models need to do more than encode characters, they need to "understand" our natural language inputs

- Standard step: take your text then encode to bytes
- The new step: bytes are grouped together into **tokens**

Tokens are fundamental "unit" of language for language models

- Important for technical, and practical reasons (£££)

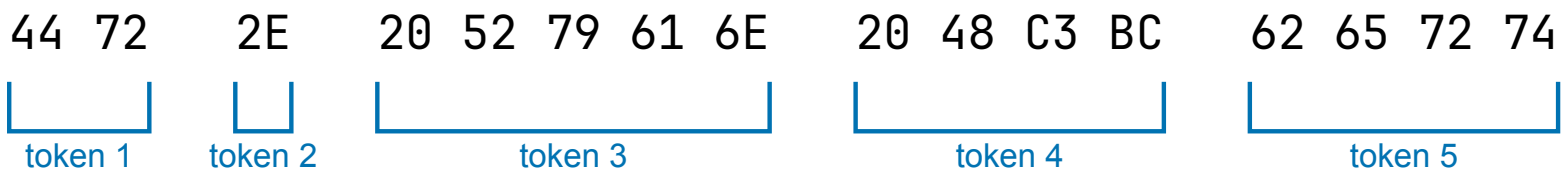
Recent addition: a new and very important "form" of digital data

Dr. Ryan Hübert

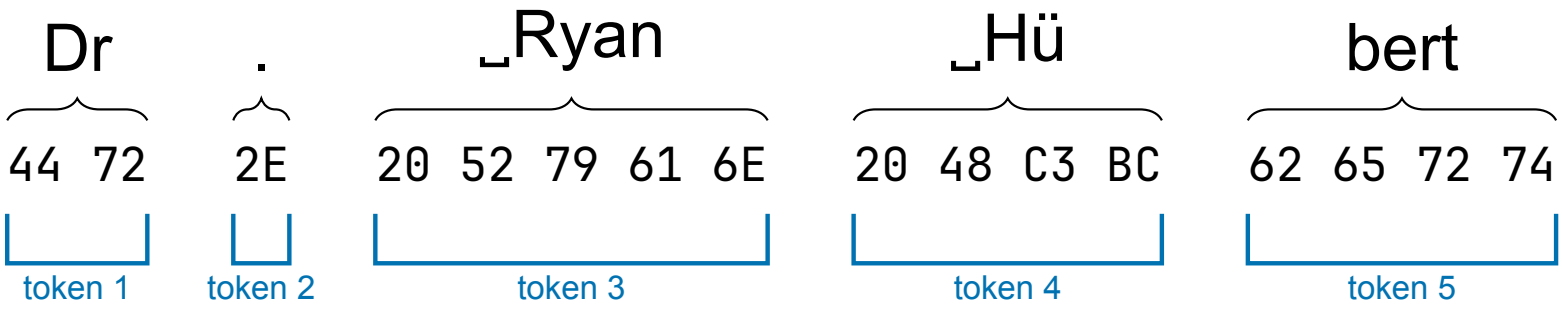
Tokens

```
44 72 2E 20 52 79 61 6E 20 48 C3 BC 62 65 72 74
```

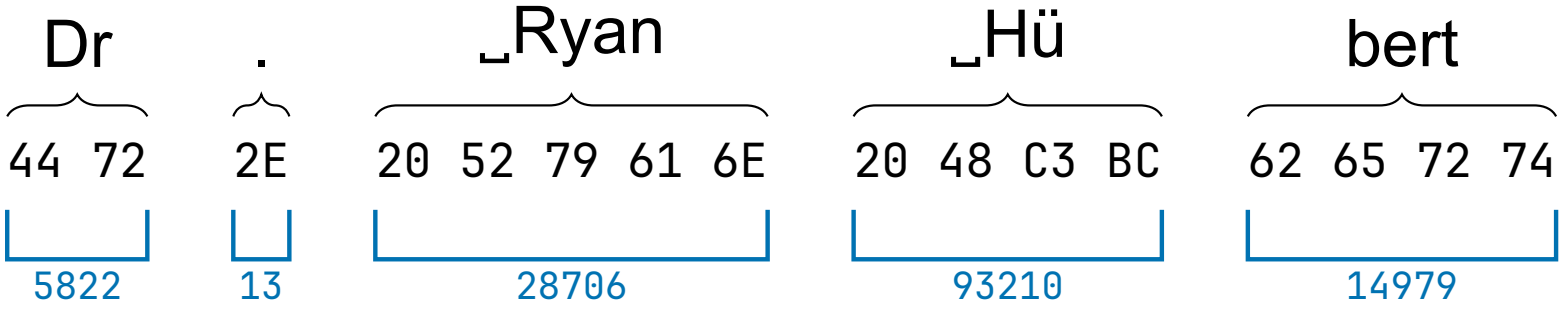
Tokens


44 72
2E
20 52 79 61 6E
20 48 C3 BC
62 65 72 74
token 1
token 2
token 3
token 4
token 5

Tokens


Dr
.
␣Ryan
␣Hü
bert
44 72
2E
20 52 79 61 6E
20 48 C3 BC
62 65 72 74
token 1
token 2
token 3
token 4
token 5

Tokens


Dr
.
␣Ryan
␣Hü
bert
44 72
2E
20 52 79 61 6E
20 48 C3 BC
62 65 72 74
5822
13
28706
93210
14979

Why not just words?

A model's **vocabulary** is its set of possible tokens

- The model is built to work with this fixed list, so even new words must be represented using existing tokens

Two options at two extremes:

1. **Whole words:** no fixed list covers every new name, misspelled word, URL, hashtag, slang, etc.
2. **Individual bytes:** can represent any text, but convey little on their own and need many tokens to represent texts

A useful compromise: group common byte sequences into tokens

- Sometimes token = word, sometimes not

Byte pair encoding

Byte pair encoding (BPE) groups bytes by frequency of use

- From bytes, repeatedly merge most common adjacent pairs
- Stop when you have a vocabulary of a desired size
- Common words become a token; rare ones are split into pieces
- Anything novel can just be raw bytes, so nothing is unrepresentable as tokens

Similar idea to UTF-8: common things are represented in fewer "units" (tokens here, and bytes in UTF-8)

- Key difference: UTF-8 is decided by Unicode, while the BPE vocabulary is measured from real-world texts

OpenAI's uses the `o200k_base` **tokeniser** (~200,000 tokens)

BPE tokenising in Python

The `tiktoken` package gives you access to the tokeniser used by OpenAI for the GPT LLMs

```
import tiktoken
enc = tiktoken.get_encoding("o200k_base")
lse_tokens1 = enc.encode("London School of Economics")
print(lse_tokens1) # stores as token IDs
```

```
[51162, 6586, 328, 56779]
```

```
lse_tokens_str = [enc.decode([t]) for t in lse_tokens1]
print(lse_tokens_str) # print the substrings
```

```
['London', ' School', ' of', ' Economics']
```

```
lse_tokens2 = enc.encode("london school of economics")
[enc.decode([t]) for t in lse_tokens2] # the text of each token
```

```
['l', 'ondon', ' school', ' of', ' economics']
```

Try it: https://platform.openai.com/tokenizer

BPE tokenising in Python

Here is a snippet of the table of tokens in the `o200k_base` tokeniser

```
for i in range(258,278):
  print(f"{i} > {enc.decode([i]).encode("utf-8").hex(" ")} [UTF-8 decoded string: '{enc.decode([i])}']")
```

```
258 > 69 6e [UTF-8 decoded string: 'in']
259 > 65 72 [UTF-8 decoded string: 'er']
260 > 20 74 [UTF-8 decoded string: ' t']
261 > 20 61 [UTF-8 decoded string: ' a']
262 > 65 6e [UTF-8 decoded string: 'en']
263 > 6f 6e [UTF-8 decoded string: 'on']
264 > 72 65 [UTF-8 decoded string: 're']
265 > 20 73 [UTF-8 decoded string: ' s']
266 > 61 74 [UTF-8 decoded string: 'at']
267 > 6f 72 [UTF-8 decoded string: 'or']
268 > 65 73 [UTF-8 decoded string: 'es']
269 > 20 20 20 20 20 20 20 20 [UTF-8 decoded string: '        ']
270 > 61 6e [UTF-8 decoded string: 'an']
271 > 20 20 20 [UTF-8 decoded string: '   ']
272 > 20 64 [UTF-8 decoded string: ' d']
273 > 68 65 [UTF-8 decoded string: 'he']
274 > 20 63 [UTF-8 decoded string: ' c']
275 > 20 70 [UTF-8 decoded string: ' p']
276 > 69 73 [UTF-8 decoded string: 'is']
277 > 61 72 [UTF-8 decoded string: 'ar']
```